SEPTEMBER 15, 2025

Coursework – Maintain and Extend a Legacy Software

MARJAHAN BEGUM AND MAX WILSON

Overview

This coursework requires you to maintain and extend a re-implementation of a classic retro game. It is worth 100%. The style of the work is designed to mirror industry practice, where you join a software engineering company, and are given things to fix with existing software, through git issues. This is the task of software maintenance.

- From Task 1-4 – You are completing software maintenance as a developer
- From Task 5-8 – You will be extending the software, with additional DevOps tools.

Like the week 3 lab, you will find yourself in a COMP2013 gitlab group in projects.cs.nott.ac.uk, and within that, a group named after your username, and within that a project with a copy of the software that needs maintenance. You should clone and work on this repository. DO NOT fork it to a new location.

During the coursework you will make versions 2 and then 3 of the software. V1 is current version (when fixed), V2 is a refactored version, V3 is where you have extended the functionality of the software.

You will receive 8 tasks, as git issues. You are expected to resolve these git issues roughly in order.

- For best practice, you should create sub-branch from the Dev branch for each of the tasks below, and merge it back to dev as you complete the git issue.
- You should git tag a commit that represents completing the git issue and make sure that tag is pushed to the server (refer to COMP1003 for reminders, see specified tag in relevant tasks).
- You should move each git issue, when completed, to the Completed column of the issues board. This will inform us that you think the task has been 'completed'.
- You may wish to record some video explanation of each task, or tak some notes, at the time of resolving the git issue, for later use Task 8.
- Do note delete the git issues.

NOTE 1: You may not be able to begin some tasks until you have been taught the relevant content in lectures.

For each assessment criteria, this module looks at **how well** you do the work. You should be making good use of gitlab features (as taught in Y1, including writing useful commit messages), using Google's java style coding conventions, creating useful content in markdown, and using Javadoc structure to comment your code.

NOTE 2: This coursework assumes the knowledge of git taught in COMP1003

After fixing V1, after completing V2 and after completing V3, you should merge dev into main (without deleting the dev branch), so that main always contains the latest version. When completely finished, you must submit a link to your video in moodle. This will count as 'submitting your coursework'.

NOTE 3: Anything that is not correctly included in or linked from the markdown as instructed for each task below will not receive a mark.

Guides

- Writing a changelog
- Writing useful git commits
- JavaFX and IntellJ

Tasks

Task 1 – Setup your repository properly and fix version 1

The project as it stands doesn't work properly. The aim of this first task is to get your git project setup properly and get version 1 working again, with good initial documentation. Clone the repository and open it in IntelliJ (or your preferred editor) and set up a dev branch to work on. Fix any compile errors that prevent the project from being built and try to run the application. Add changes made to Changelog.md (you should keep the changelog updated throughout the coursework with dated updates describing changes to the repository). Redo the primary README (as below). Commit these to the project and push to projects.cs.nott.ac.uk (herein called 'the server').

Deliverable:

- Properly setup repository for working version of V1 and initial documentation.
- Add a screenshot of the software running with no compile errors to the git issue.
- Link to the changelog added to the main README
- Git-tag with "v1", and push it to the server.

Guideline for maintaining the main README

- Project Title + Author Name
- Brief project description
- How to install and run the project
- How to use the project (i.e. how to play the game)
- Credits [if any third-party element were used]
- Links to other key markdown files

Task 2 – Understanding the legacy code and plan version 2.

Read and try to understand how the current software is working, and add code comments. You may wish to begin to learn javadocs. Use any UML diagram of your choice to visualize and communicate how the code currently works. List your initial thoughts on issues in the code. Think about how you would like to restructure it, so that the quality of the code is improved, and explain this redesign with another UML diagram of your choice. Add this content to SoftwareDesign.md under the Version 1 and Version 2 section.

Deliverables:

- Markdown documentation (500 words, and 2 diagrams) with the diagrams embedded that 1) describes the structure of v1, and 2) describes the planned refactoring for v2.
- Updated code with comments for v1.
- Link to the Software Design markdown added to main README
- Git-tag with "task2"

Task 3 – Create tests for v1 and for restructured code.

Create regression tests for v1, that you can use to show that any refactored version still works. Create initial tests for your redesign, initially with some stub methods if needed. You may wish to investigate Maven to understand how the tests are being run.

Deliverables:

- JUnit files with code comments for the tests
- A test plan table in Testing.md explaining the purpose of the tests
- Git-tag with "task3"

Task 4 – Maintain the software by applying refactoring techniques and principles

Objectives:

- Restructure the code into meaningful packages
- Move the software from swing to the chosen new interface framework JavaFX
- Comprehensively test the code with unit tests and integration tests.
- Restructure the code into a MVC architecture (programmatically or via FXML).
- Refactor the code to incorporate relevant design patterns.
- Refactor the code to follow good OO, SOLID, DRY, KISS and CUPID principles where its relevant. There are issues in the code that does not adhere to these principles.
- Eliminate code smells using the refactoring techniques. The following code smells already exists in the code base:
    - Long method, Extract class, Move Function, Remove dead code, Parametrized functions, replace conditional with Polymorphism, Decompose conditional.
    - Think about ways of restricting the code to reduce LOC through better data structures.
    - You may identify other code smells and eliminate the code smell.

Deliverables:

- V2 of the code with comments now as Javadocs
- Markdown documentation (in SoftwareDesign.md) for how the code was refactored, with cross references to git commits (500 words)
- Markdown documentation (in SoftwareDesign.md) describing the final structure, using e.g. UML (250 words, 1 UML diagram)
- Completed test table in Testing.md
- Git-tag with “v2”

Tasks 1-4 should be completed before moving on to Task 5

At this milestone in the maintenance of the software you should be at a stage where the software is working as it was **originally intended** to with no additional features. Task 5 onwards are for software evolution.

Task 5 – Design/plan additions to the software

Plan three additional features to the software that you intend to include in V3. This may involve additional classes or extending an existing classes/methods. You may wish to consider any of the following ideas, whilst remembering that the work should be able to demonstrate doing evolution “well” for the assessment criteria:

- A start screen with colour theme choice, using JavaFX, allowing for preference options (color scheme, sound etc.), or for specific accessibility requirements
- Additional in-game Control Buttons - Start/pause/end (to take back to the main menu)
- Add scoring
    - Scoring mechanism
    - Persistent score boards
    - Display current high scorer and their high score
- Add additional obstacles/enemies or add additional maps to the game
- Add additional difficulty levels
- Any additional new features from your own ideas

Deliverables:

- Documentation in SoftwareDesign.md including embedded lo-fi prototypes and/or UML, of the new features (500 words, plus reasonable number of diagrams).
- New Test Plan (in Testing.md) for the planned features
- Git-tag with “task5”

Task 6 – Configure integration (CI/CD) pipeline

CI/CD pipelines are useful for ensuring that your code is automatically built and tested at a regular intervals. This task will involve configuring a pipeline using a .gitlab-ci.yml file.

So far, all it does is run junit tests for us. The exact details of this task will be released later in the semester.

Deliverables:
- Markdown in Testing.md explaining the configurations (500 words)
- Updated YAML and configuration files
- Git-tag with “task6”

Task 7 – Extend the software for the new features, using the pipeline

Refactor your code to include additions you planned in Task 5. The objectives for code quality are the same as Task 4. Your code should not have any code smells, should follow coding conventions, should use good coding principles (OO, SOLID, DRY, KISS and CUPID), should make use of relevant design patterns, and be well tested using test-driven development. Use appropriate design patterns during this refactoring.

Deliverables:
- V3 of the software (including updated Javadoc)
- Revised documentation in SofwareDesign.md (500 words)
- Git-tag with “v3”

Task 8 – Video justifying your decisions in the coursework

Note: this issue does not need a branch, etc. but the git issue should still be moved to completed when complete.
Create a 6-minute video in the following sequence (with indicative lengths in seconds):

- Run the game to show that it builds, show different features you have added and the associated new code (90s)
- Justify the final architecture of your software (30s)
- Explain how your code meets one or more coding principles (e.g. OO, SOLID, DRY, KISS) (60s)
- Describe the 3 most important ways that you refactored the software (60s each). For each, cover:
    - Explain why it was important
    - Explain the code change
    - Why you did it that way
    - If relevant, explain if you considered a different way to approach it

The format should be .mp4, and the video must include a view of your face in the recording. Be mindful of video size – it should be high enough resolution for us to read what you are showing (e.g. HD), but should not be too big to hog storage. Max’s recommendation is to start a Zoom call with just yourself, screen share what you want to share, make sure camera and mic are on, and hit record (to local storage). However, many other approaches are available. You may wish to use a video editor approach if you are combining audio with multiple screen captures. Handbrake is then a good free app that is good at compressing videos while maintaining HD. You should make sure your video can be understood clearly – you may wish to embed subtitles in the video for clarity. Marks may be lost if we cannot understand what you are saying.

Deliverables:
- Store the video in your onedrive, create a onedrive link share that is open to anyone in the university (see guide at end of document), and submit this link in full to moodle.
- Paste the same link as a comment to the git issue and close it.

Deadline

8th January 2026 3:00pm

This deadline is for the final completion of all it issues, and uploading a final video link to moodle. You should complete most git issues earlier than this. Each git issue provides an indicative due date to stay on course.

Assessment Criteria

We **recommend** dedicating approximately 100 hours to the coursework. Please keep in mind that the skill level varies quite a bit in a large class of students, and that the exact number of hours depends on your individual skill level.

Criteria	**Weight**	**Relevant Tasks**	**What counts as meeting this criteria "well"**
Use of git features	5%	All	The repository is well organised and clean. Professional branching strategies are consistently applied. The project makes exemplary use of issues (including adding useful comments to issues being resolved), milestone and labels. Git commits are done regularly and in structure that adheres to a professional approach.
Quality of code review	5%	2	Clear and concise explanation in the markdown documentation that aligns with the diagrams, and useful Javadoc. Diagrams are clearly labelled and contain evidence of understanding of the current structure.
Quality of redesign	5%	2, 5	Clear and concise explanation in the markdown documentation that aligns with the diagrams and git features. Diagrams are clearly labelled and there is evidence of design of new features in the UML diagrams.
Quality of refactoring	40%	4, 7	Actual code changes are implemented well, according to code conventions and taught principles, documented in Javadoc and comments within the code. Changes are clearly explained and usefully cross referenced with git commits in the markdown.
Quality of testing	10%	3, 5, 7	Initial test plan, completed test plan, and tests are well organised and usefully cross referenced with git commits. Test plans and tests are aligned with taught content. The tests should have >80% code coverage. There is clear evidence of CI later in the project.
Quality of documentation	5%	All	All the documentation (markdown and javadocs) is clear and concise and well organised, and effectively communicate the content specified for each of the tasks.
Configuration of infrastructure	10%	6	The CI/CD is purposefully designed to be useful (and is evidently used in the coursework in the later stages of the project), and explanations of this structure in markdown show clear understanding and implementation of CI.
Reasoning for decisions	20%	8	A quality .mp4 video with clear explanations (including views of code) and justifications of all the bullet points under this task. The video must not be more than 6 minutes long. It must be accessible by the teaching team (follow the guide at the end of this document). Good marks come from clear explanations and justifications of your decisions in the project. No marks will given for just showing code and features without explanation.

Resolving git issues early – 5% Bonus

In industry, speed of work is considered as well as quality of work. The criteria above focus on quality. In addition to these criteria, students are able to earn a 5% bonus by submitting V3 of the software (git-tag) before the end of term (Dec 12th, end of day). Any further commits after Dec 12th would remove this bonus.

Word limits specified in the tasks

In industry, staff are rewarded for providing the most salient information in the most concise and clear way, clearly justified. The provided word limits (and video length) are there to help you learn to communicate important information concisely to us. Word limits are considered as part of doing each task well.

Use of ChatGPT and other LLMs

This coursework follows the university policy for use of ChatGPT and other LLMs. This includes not submitting LLM generated content as your own for assessment (false authorship). However, you are allowed to use LLMs to assist your learning and understanding of concepts and learning outcomes.

For this coursework, you **may not**:

- Use e.g. co-pilot to write code
- Use LLMs to generate any code that will earn marks
- Use LLMs to generate documentation that explains code structure

For this coursework, you may:

- Ask LLMs to explain code to you
- Ask LLMs to help debug a problem
- Ask LLMs to help you understand how to use tools like yaml, DevOps tools, gradle, or junit, etc.
- Ask LLMs to reduce/optimise the text you created for markdown files

If you do use LLMs for these purposes, you are required to submit a log of your interaction with those LLMs to moodle as a PDF. If you do use any LLM-generated code, you are required to highlight which code was authored by an LLM in the same way as citing the source of code copied from the internet, so that it is not considered when your project is being marked.

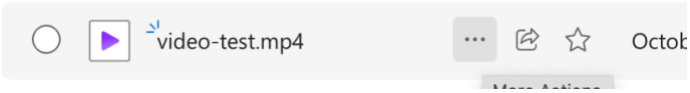
Plagiarism Vivas

The module convenors reserve the right to require that a sample of students attend in-person to explain their code, either by random sample, or for cases of suspected plagiarism.

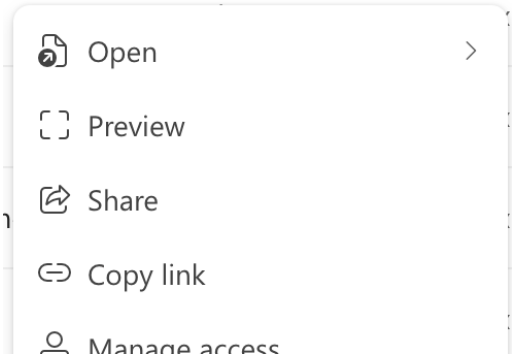
-- End of Coursework Description --

Correct video sharing guide

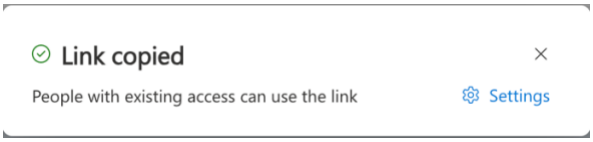
1) Store your video in your OneDrive. Right click on your video, or press the […] if viewing through a browser



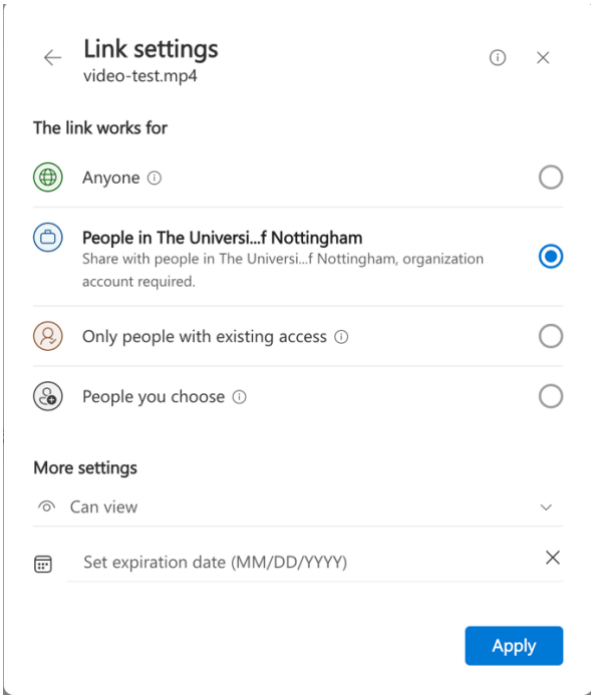
2) Select ‘Copy link’



3) This popup will appear, Click on Settings.



4) You will get the following pop-up, and select ‘People in The University of Nottingham’. And click ‘Apply’.



5) It will return to this view, where it will have auto-copied ready to paste into moodle and the git issue.